



Gen 6 Motor Controller Control Loop

The contents of this presentation are confidential and are not to be shared without permission of the author. All rights are reserved.

A word on notation and how time, speed and acceleration interact

Conventional
Notation

Differential
Notation

Newtonian
Notation

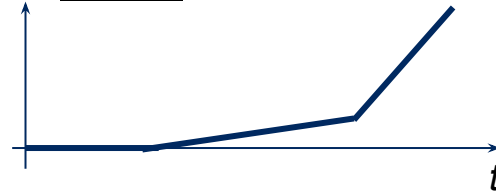
Example
Motion

Distance

x

x

x



Speed

v

$\frac{dx}{dt}$

$\dot{x}$



Acceleration

a

$\frac{d^2x}{dt^2}$

$\ddot{x}$



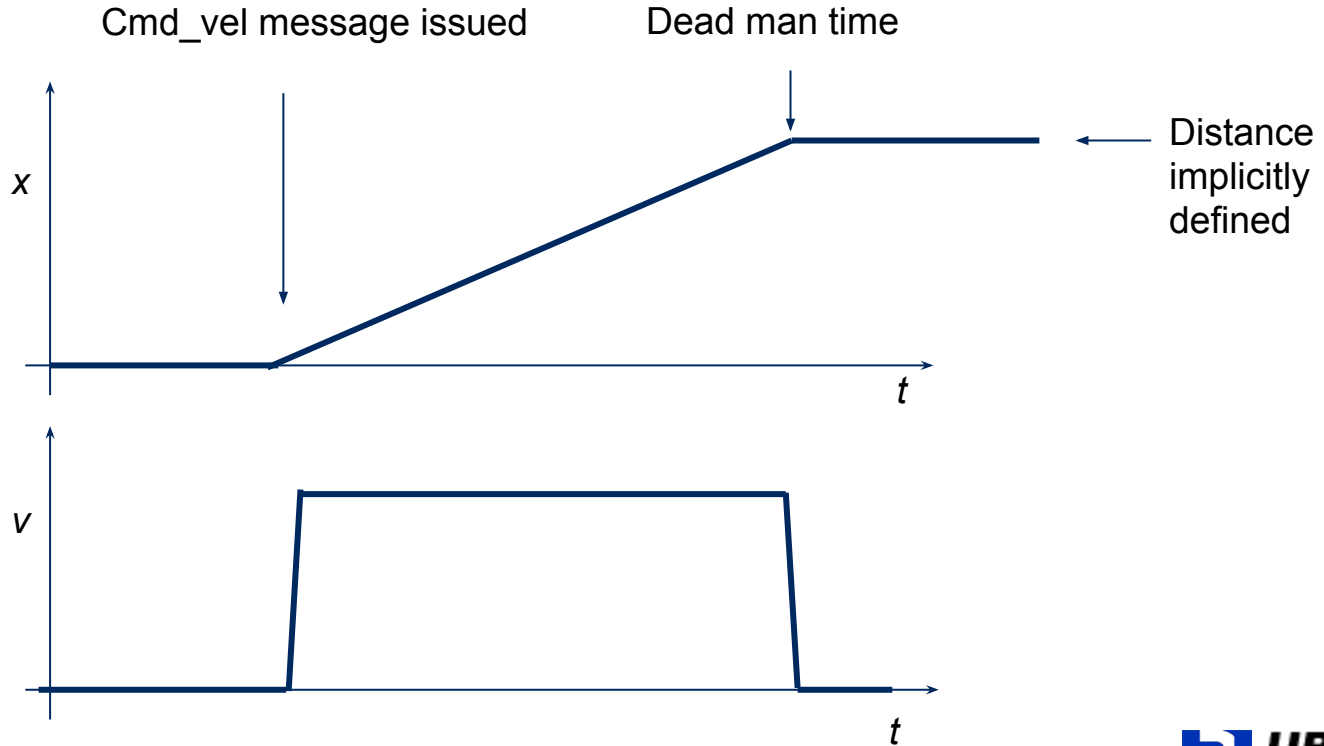
If you define 2 out of speed, distance and time the remaining value is defined by implication (to first order)

$$vt = x$$

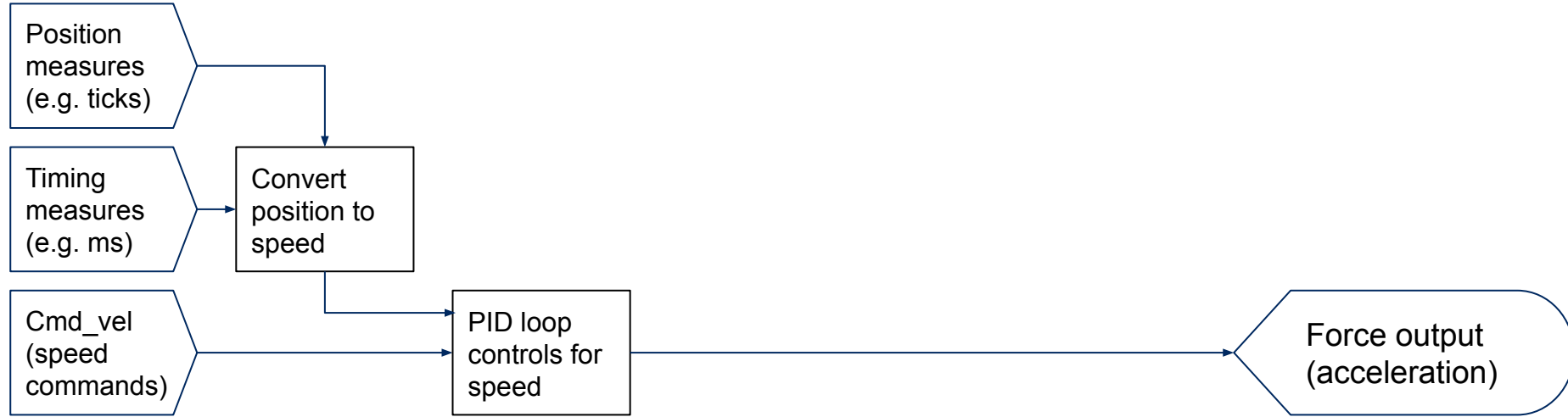
$$\frac{x}{t} = v$$

$$\frac{x}{v} = t$$

`cmd_vel` gives us speed, and defines a “dead man” time
distance is thus implicitly defined

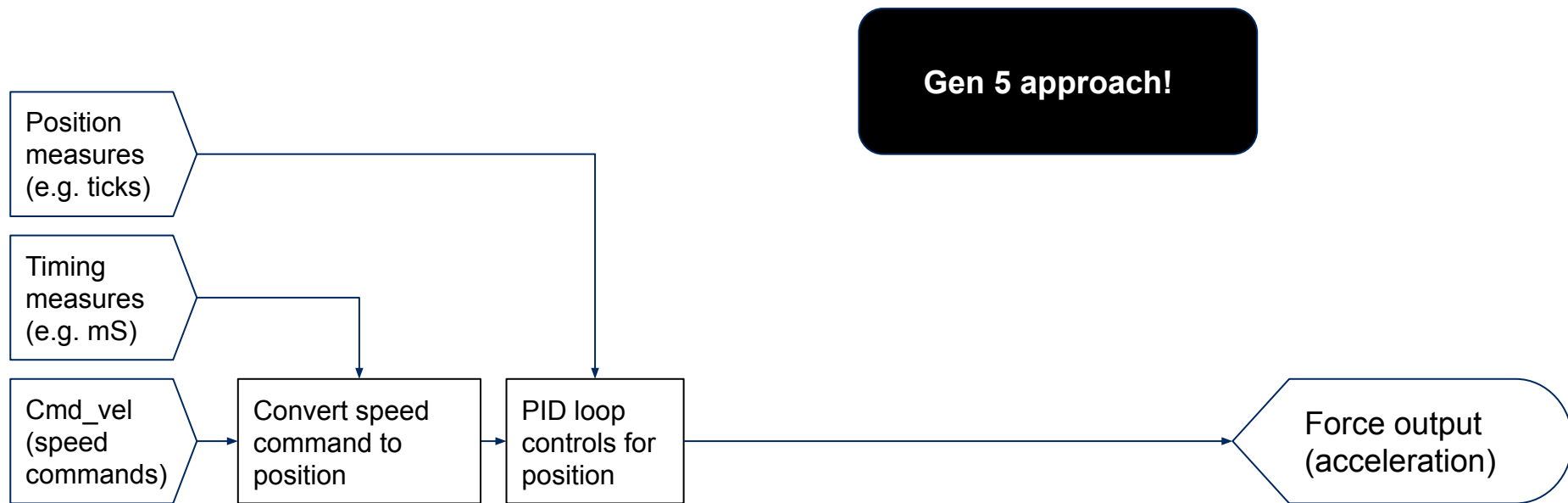


Usually a speed based PID loop controller is used



Note: In any given magnetic field the force in a wire is proportion to the current it carries. As such for our motor the force output will typically be something that controls current in the motor - for example a PWM. In our FOC system code I expect the code contains some variable that equates to force.

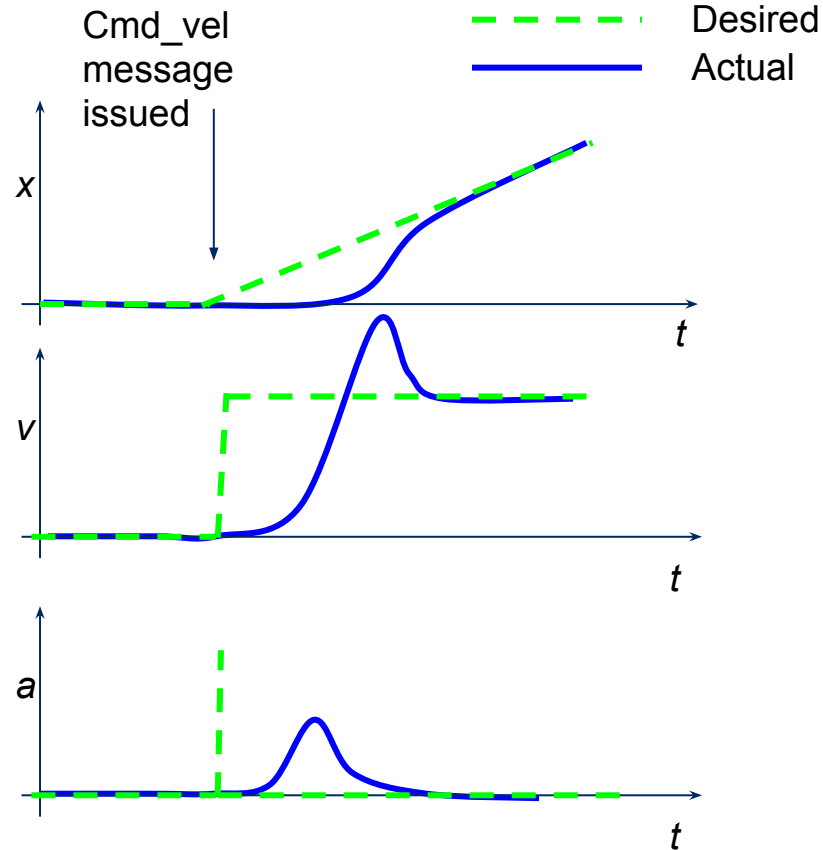
We found using a position based controller gave better results



Timing measures update way faster than position measures. So its usually better to estimate target position and compare this to the actual position than it is to wait until you have a position update to estimate the speed. In this way your control loop can run faster than the slowest likely rate of position updates. If you use speed as a metric, your control loop usually has to run much slower than the slowest rate of position updates.

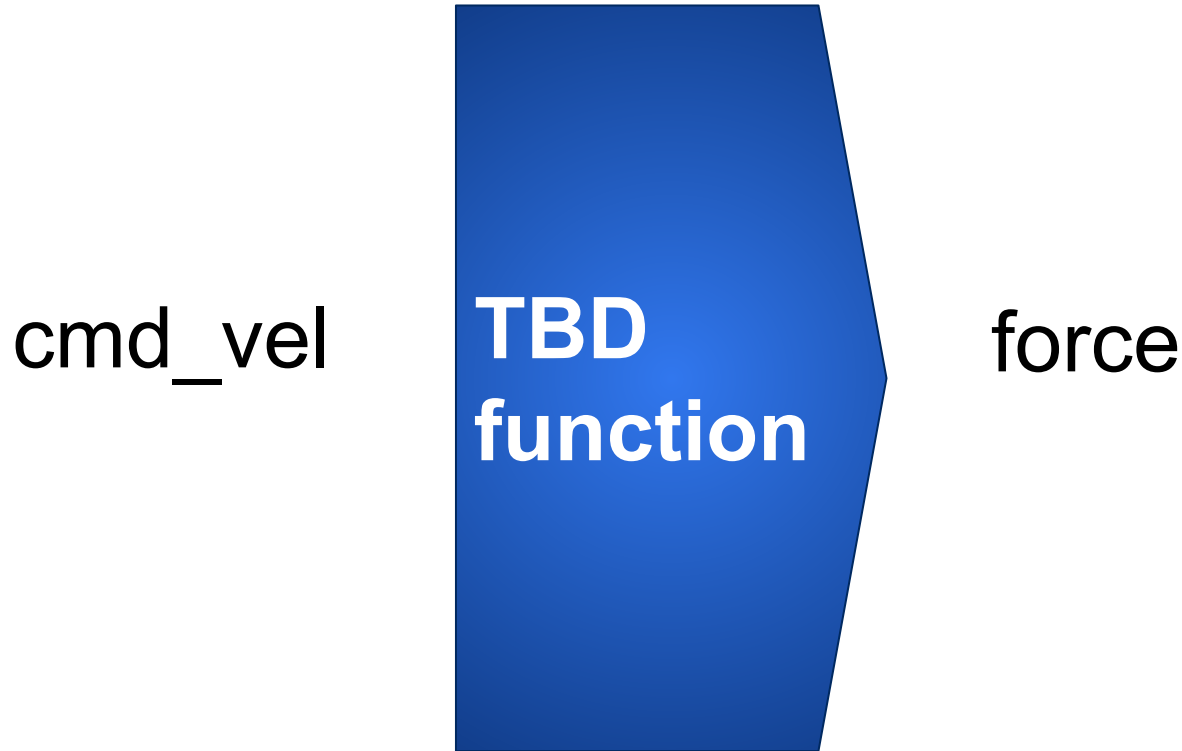
This approach is not without its problems

After the `cmd_vel` message is issued the controller has to wait until the motor is out of position to generate any force. This takes time



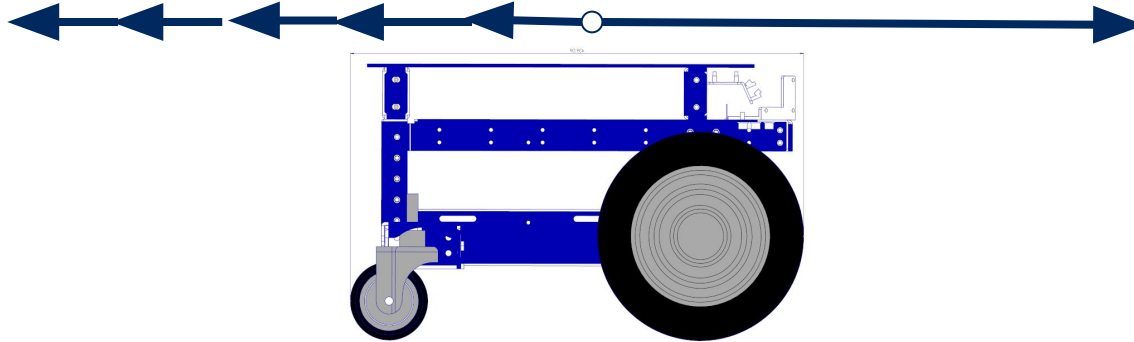
Position based control results in a less responsive controller

A method to anticipate the force needed based on a new cmd_vel command would increase responsiveness



The amount of force needed to accelerate or drive an object is subject to physical laws

Static Friction (C_s) Rolling resistance (C_r) Linear Friction ($C_f v$) Drag ($C_d v^2$) Momentum Force (ma) Force (F)



We can devise a simple physical model that encapsulates the need to accelerate for a given time to achieve a change in velocity

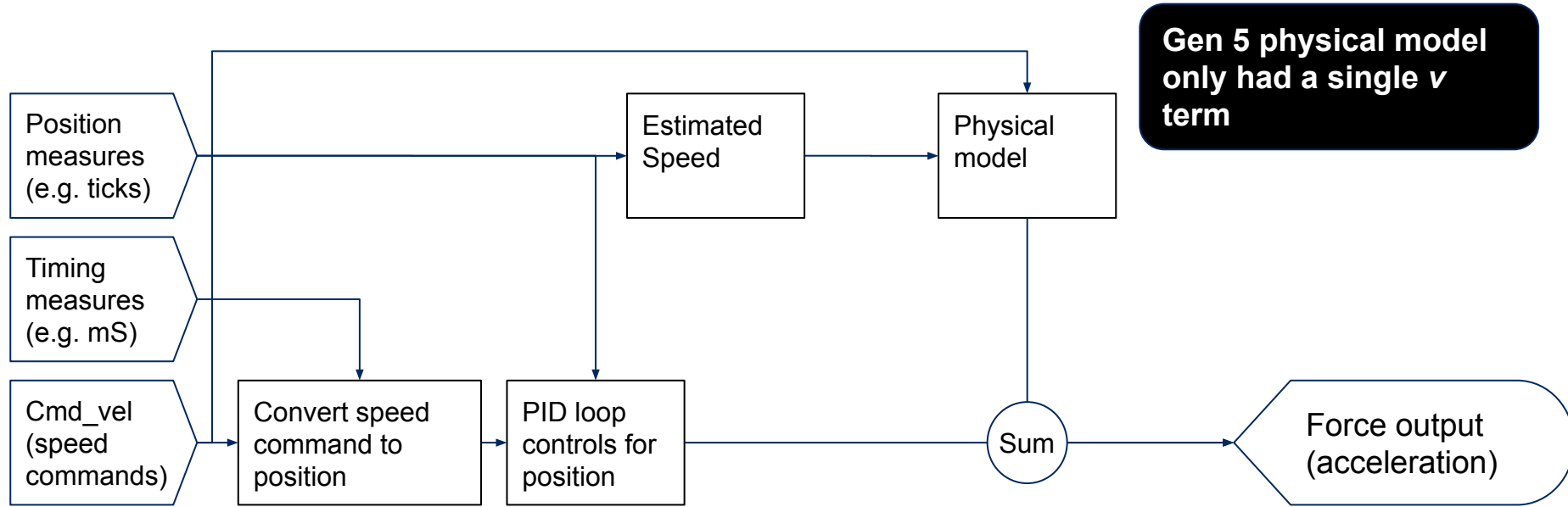
The same model can estimate the force needed to overcome friction, resistance & drag

This physical model would reduce reliance on PID terms to achieve changes in velocity

The better the physical model the less the PID terms would be used

- Static Friction (C_s) is applied when $v=0$ and $v_{target} \neq 0$
- Rolling Resistance (C_r) is applied when $v_{target} \neq 0$
- If you need to accelerate because new commanded velocity is different to a previous one you need to define the amount of acceleration and duration this requires 2 parameters in a physical model noted as A_F & A_d

Adding in a physical model makes it more responsive



Pseudo code for physical model for Gen 6

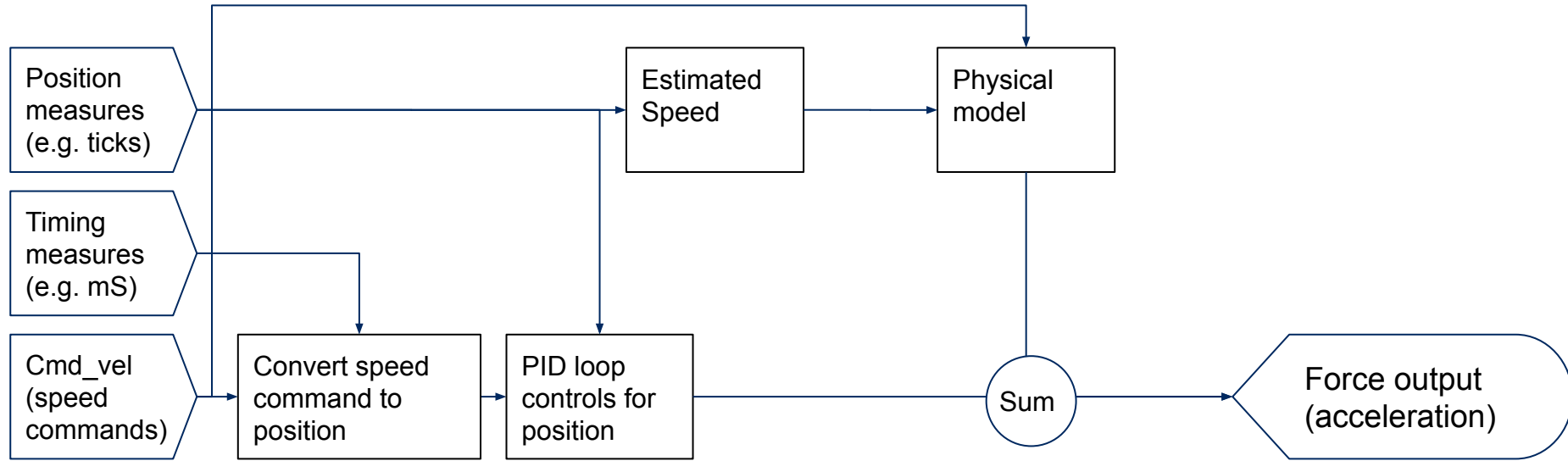
Physical (current_time, command_received_time, commanded_velocity, previous_commanded_velocity, current_velocity, estimated_velocity_error, Param_Ad, Param_AF, Param_Cd, Param_Cf, Param_Cr, Param-Cs)

```
{
  Int cvsign=0;
  If (commanded_velocity>0)                                     //need a sign correction for later terms
  {
    cvsign=1;
  }
  else {
    cvsign=-1;
  }
  int Force = 0;
  int acceleration_time = 0;
  acceleration_time = Param_Ad * (commanded_velocity - previous_commanded_velocity);
  if (acceleration_time>=(current_time - command_received_time)) //Estimate duration of acceleration
  {                                                                //Evaluate if it should be accelerating
    Force=Param_AF;                                              //Accelerate according to acceleration force
  }
  if ( (abs (current_velocity) > estimated_velocity_error) && (commanded_velocity!=0)) //Evaluate if its moving and is supposed to be moving
  {                                                                //If its moving add in force to overcome rolling resistance
    Force=Force + cvsign * Param_Cr;
  }
  else {
    Force=Force + cvsign * (Param-Cs+ Param_Cr);                //If not moving add in force to overcome static friction
  }
  Force = Force + cvsign *Param_Cd * commanded_velocity ^2;     //Add in force to overcome drag
  Force = Force + Param_Cf * commanded_velocity;                //Add in force to overcome linear friction
  return Force;
}
```

Notes on proposed physical model for Gen 6

- Does not attempt to minimize jerk ($\frac{d^3x}{dt^3}$) or higher order terms. This could lead to unwanted vibrations. Consider using a gaussian acceleration profile rather than a square one to provide appropriate impulse while minimizing all higher order terms.
- The system will achieve the best adherence to target position with high `param_acceleration_force` but this will also have the highest jerk
- The drag term is assumed to be the same forward and backwards and in rotation - this is certainly not true but may be a good approximation.
- Does not take in to account angular momentum at all. This may mean that it gives sub-optimal results if it is tuned with a rotating robot and used with a robot that drives in a straight line and vice versa. Consider working out the 3 body dynamics for a differential drive robot for a more accurate impulse estimate. Given we probably care more about rotational accuracy than linear accuracy, we may want to tune for rotation.
- Assumes the time between velocity commands is large as compared with the acceleration time.
- Obviously use system clock rather than passing a “`current_time`” variable
- All ***param_XXX*** variables should be exposed for tuning

Linear term PID loops are limited



**The PID assumes
linearity - which is
often not the case**

Extended PID loop

PID loops typically calculate an error and minimize it. In our case we are controlling for position so the error is:

$$E = x_{target} - x_{actual}$$

Where:

$$x_{target} = x_{targetatcommandtime} + t_{sincepriorcommand} v_{command}$$

In order to create the ability to more effectively deal with non-linearities in the system, and because we ultimately want to minimize E^2 not E . We propose to extend the PID loop as follows.

	<u>First Order</u>	<u>Second Order</u>	<u>Third Order</u>
Proportional	$P_1 E$	$S_P P_2 E^2$	$P_3 E^3$
Integral	$I_1 \sum E$	$S_I I_2 (\sum E)^2$	$I_3 (\sum E)^3$
Differential	$D_1 \frac{\delta_w E}{\delta_w t}$	$S_D D_2 (\frac{\delta_w E}{\delta_w t})^2$	$D_3 (\frac{\delta_w E}{\delta_w t})^3$

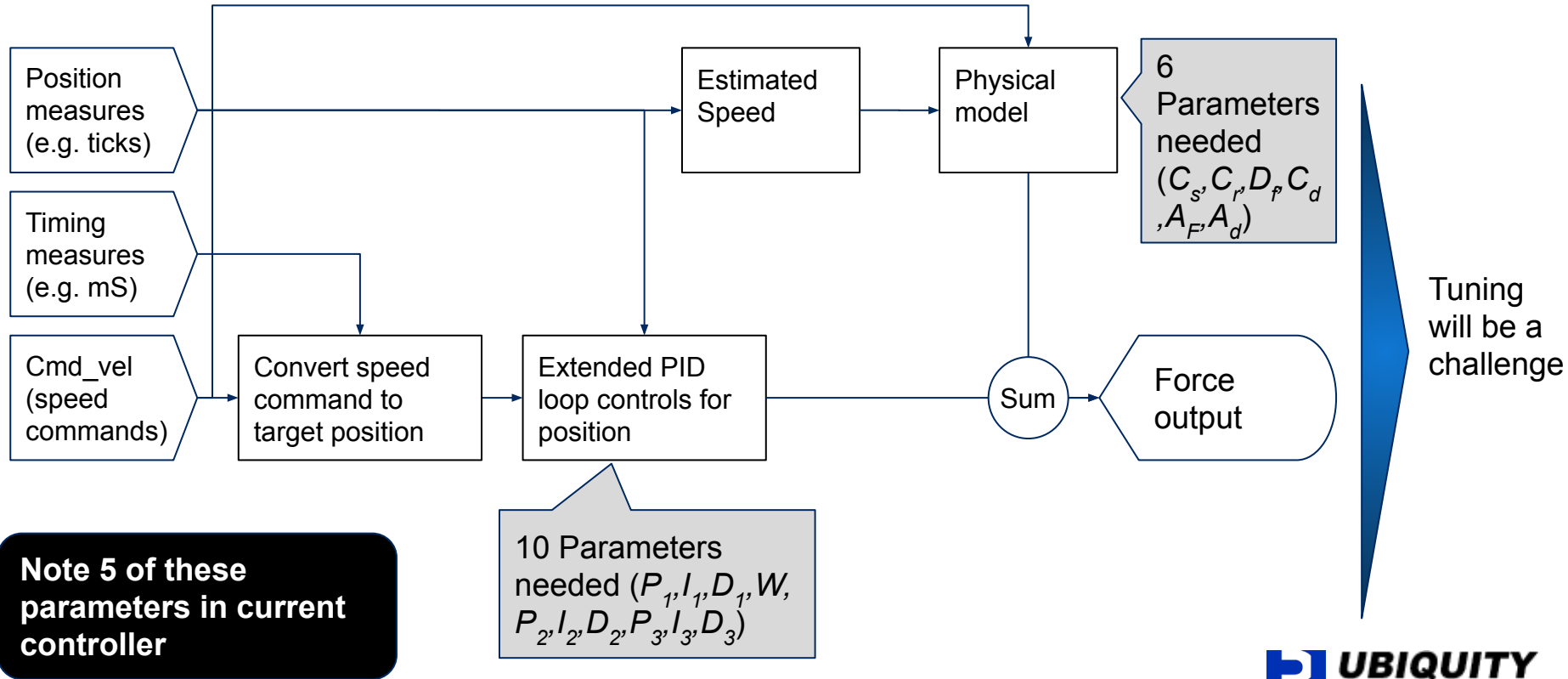


Tried and known to be
useful on Gen 5

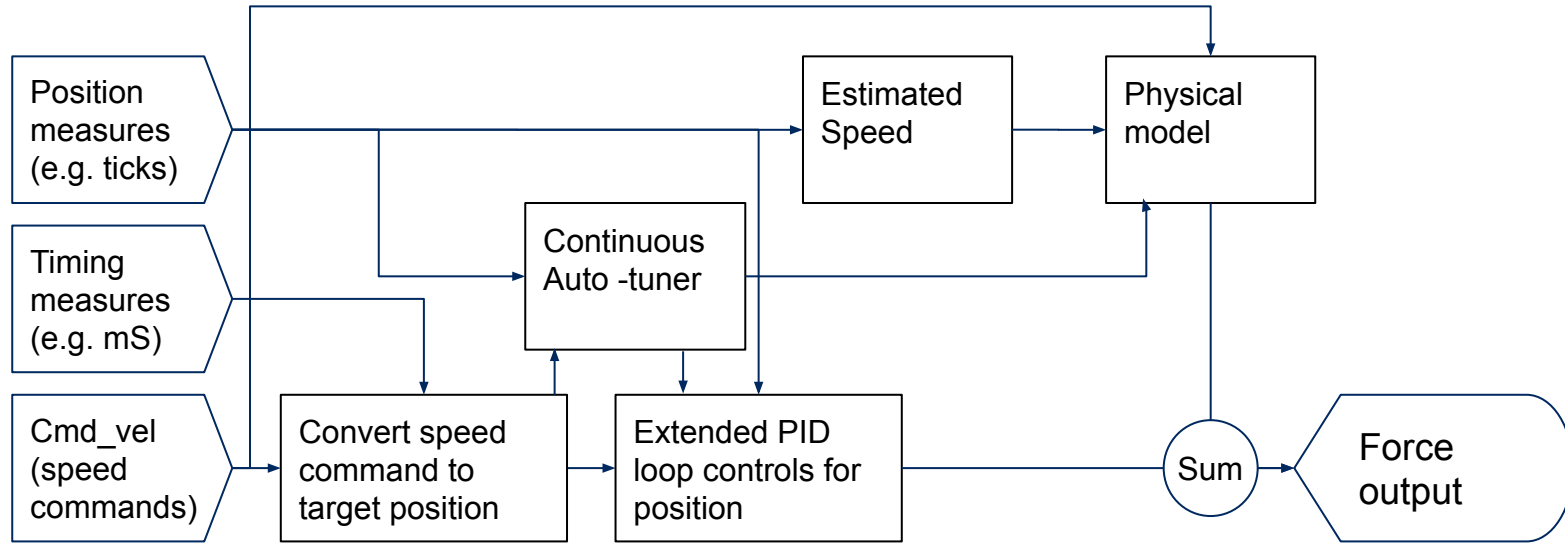
Note: δ_w is the difference over w cycles of the main PID loop. Typically if you use only a single loop the differential term is too noisy

Where S_P , S_I , S_D is the sign of the error E , Integral of the error and the Differential of the error accordingly

This control loop requires many parameters - 16 for current approach



We propose a continuous auto-tuner to optimize all the parameters



The continuous auto-tuner would minimize $\sum E^2$. By taking samples of $\sum E^2$ over some period of time t . Then using a quai-Newton or similar to find the minimum for the parameters.

Why minimize E^2 ?

- Its a provable point that this corresponds to a critically damped system - e.g. the system that most quickly returns to target with minimal overshoot
- Least squares minimization implies maximum entropy for a gaussian system - as such it is the estimation that makes the fewest assumptions and most perfectly encapsulates occam's razor for any given data set

Proposed Auto Tune Approach (first time tune)

- 1) Set all parameters to zero except P_1 that will be set to some low number (e.g. 1)
- 2) Set t , the number of samples such that it collects say 2 seconds of data
- 3) Cause the robot to drive in random 0.5 second bursts
- 4) Use quasi-Newton method to optimize P_1
- 5) Use quasi-Newton method to optimize $P_1, C_s, C_r, D_p, C_d, A_F, A_d$
- 6) Use quasi-Newton method to optimize $P_1, C_s, C_r, D_p, C_d, A_F, A_d, I_1, D_1, W$
- 7) Use quasi-Newton method to optimize $P_1, C_s, C_r, D_p, C_d, A_F, A_d, I_1, D_1, W, P_2, I_2, D_2$
- 8) Use quasi-Newton method to optimize $P_1, C_s, C_r, D_p, C_d, A_F, A_d, I_1, D_1, W, P_2, I_2, D_2, P_3, I_3, D_3$

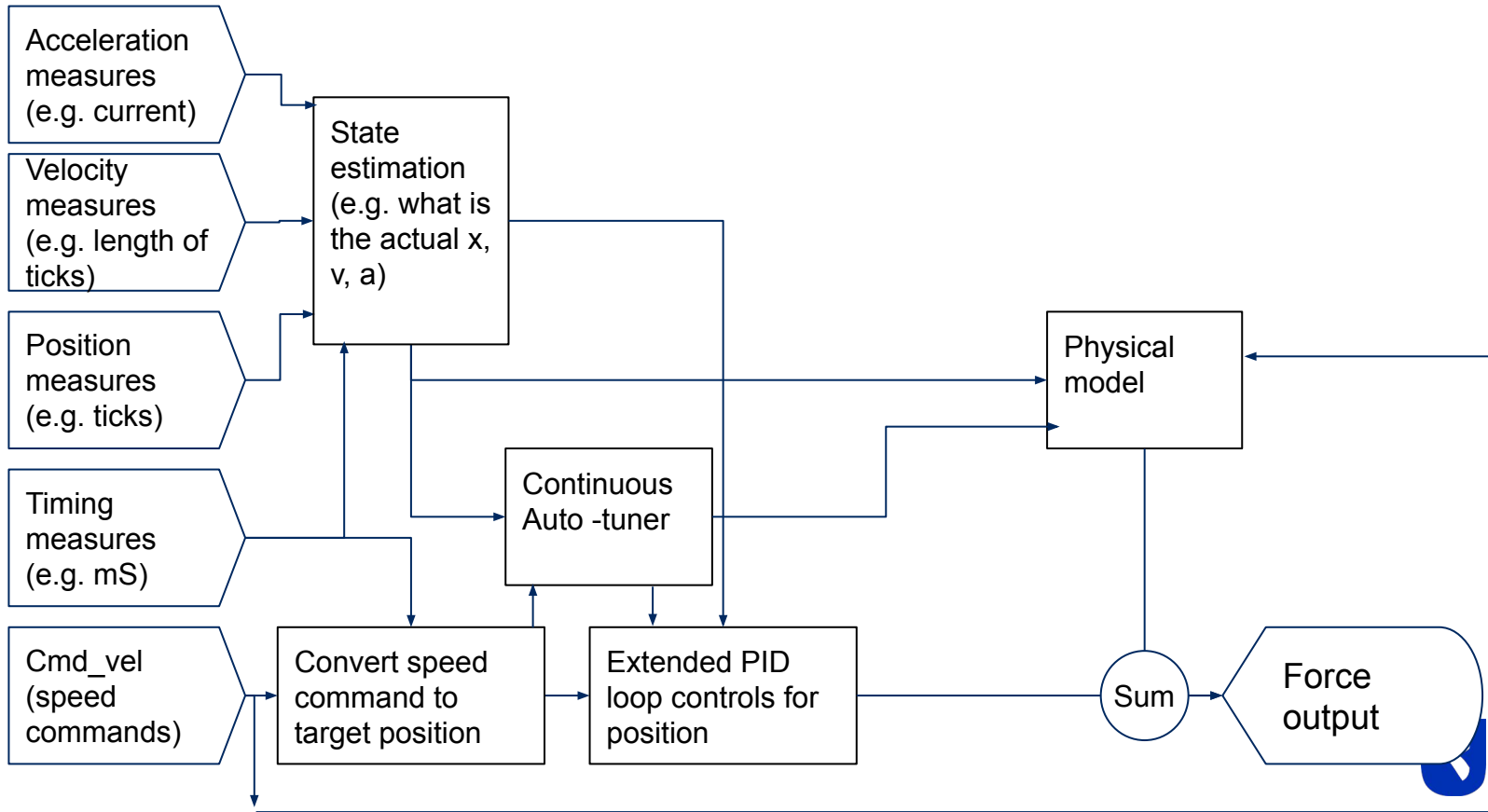
Note A_F and W have maximum permissible values

Proposed Auto Tune Approach (continuous tune)

- 1) Set all parameters to first time tune parameters
- 2) Set t , the number of samples such that it collects say 2 seconds of data
- 3) Use quasi-Newton method to optimize $P_1, C_s, C_r, D_r, C_d, A_F, A_d, I_1, D_1, W, P_2, I_2, D_2, P_3, I_3, D_3$
- 4) Collect the maximum E^2 and the average E^2
- 5) If maximum E^2 is larger than is expected given the number of samples collected in time t and the average E^2 make t larger (exact math on this to follow) - goal is to come up with an entropy maximizing ensemble of E . E will most likely have deviations in large accelerations and decelerations these occur on startup and stop. They are relatively rare.
- 6) Some method to reduce t that is TBD but needs to be such as to avoid local overfitting

Note A_F and W have maximum permissible values

Ok but we have multiple position, velocity and acceleration measures so we actually need to do state estimation



State estimation

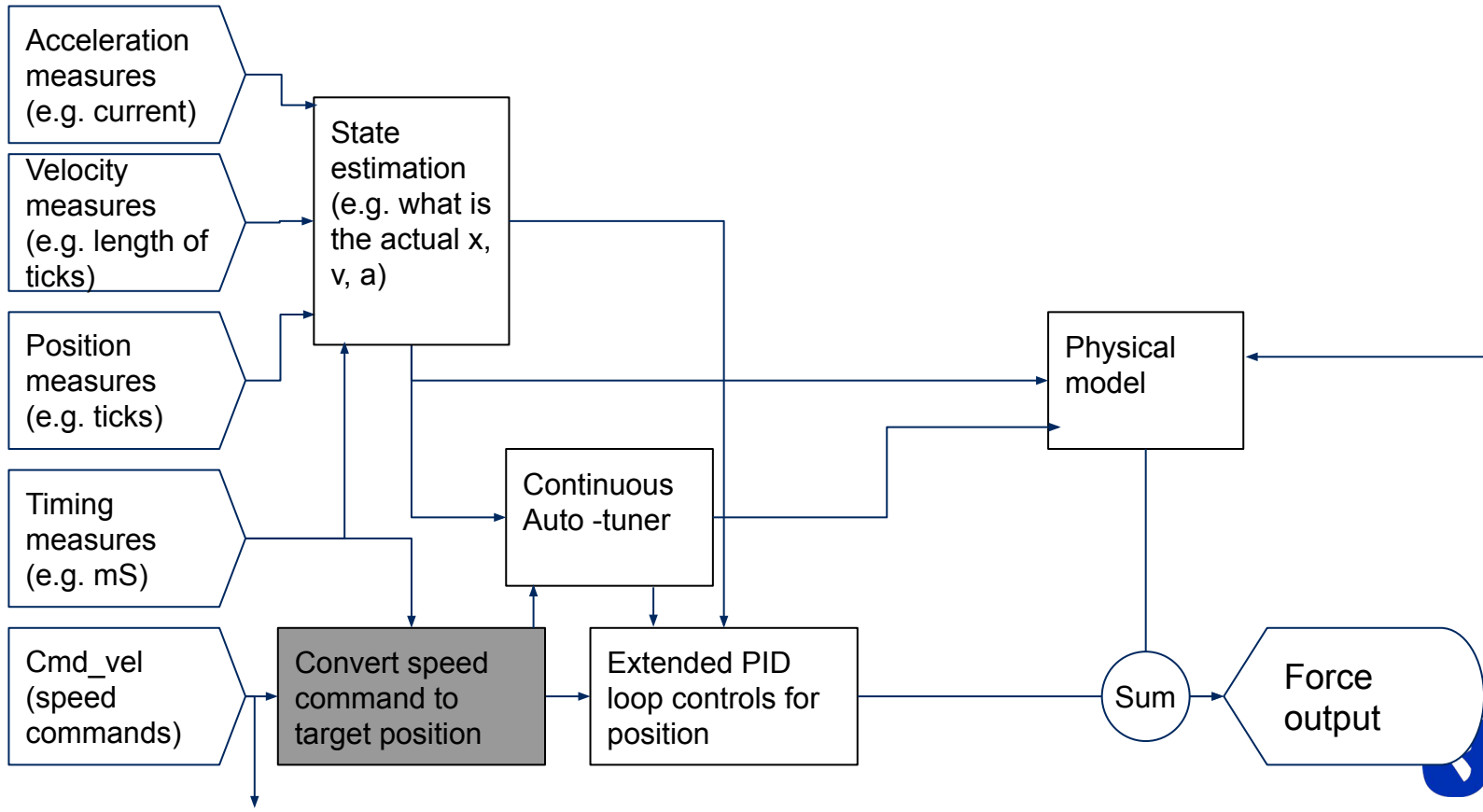
The purpose of state estimation is to take a variety of different measures and then estimate the most probable system state (e.g. speed, position, etc.) from a set of data that may be noisy. Propose using a metropolis-hastings like particle based approach. This relies on a highly parameterized state combination that takes in all the data and combines them based on a parameterized weighting and generates an output number + error. Obviously the input weighting corresponds to an error parameter.

Its helpful to put the parameters in a hilbert space, to reduce dimensionality (e.g. a space filling curve that maximizes locality) so your n dimensional space is reduced to a 1 dimensional space (e.g. a line).

This is what the algorithm would do.

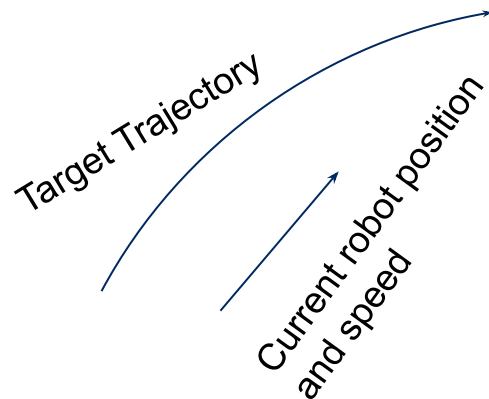
- 1) Start with an initial state guess of the parameters - use this to estimate the output
- 2) Select several different sets of parameters that differ randomly from the initial guess we will call each one of these sets a particle
- 3) For each particle accumulate the entropy estimate over some period of time
- 4) Find the particle with the highest entropy estimate use that to estimate the output from now on
- 5) Eliminate the particles with a low entropy estimate
- 6) Select a new set of particles further in the direction of the hilbert line with a dispersion based on how similar the particles are (can use a “simulated annealing method” to decide what that is)
- 7) Go back to step 3 again

Can we refine how we calculate the target position?



Can we refine how we calculate the target position

Instead of describing a target velocity, why don't we describe the relative position and speed of the robot relative to the target trajectory



The motor controller would then execute an action to close the gap on the current trajectory while respecting acceleration limits

Control loop

Can we simply use a topological argument to define the formula for return to an ideal trajectory.

Extended FOC control

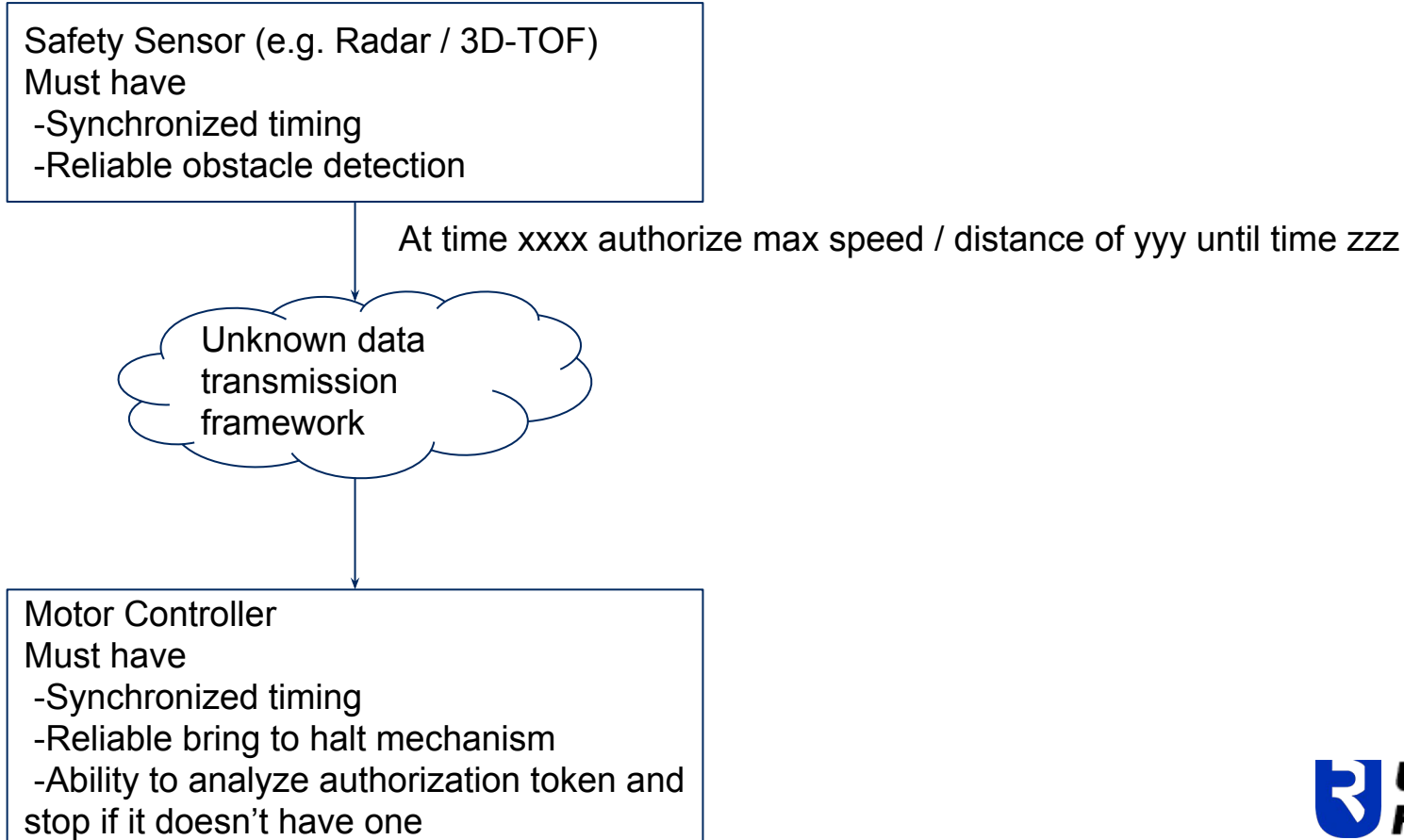
Use maximum entropy state estimation and extended PID loops to control the current in the field oriented control loops.

This idea models that presented here:

<https://patents.google.com/patent/US8981702B2>

But clearly uses maximum entropy methods to tune the FOC control loops rather than bandwidth oriented methods.

Safety authorization Mechanism



Other ideas

Use force feedback to determine if a collision has occurred and if it has then stop the robot, in practice this would occur as an estimate based on fusing

- 1) A deviation in the error number in the position controller for a well tuned physical model that also includes going up ramps
- 2) A deviation between the estimated force needed from the physical model and the actual force measured from current measures

Use similar measures to enable an “power assisted push” mechanism. In short if force is being applied to the robot the robot should amplify and help. This is both for safety and convenience. The robot, in effect, should amplify any external force that is being applied to it.

Proposed order of work

- 1) Add accelerometer to improve state estimation
- 2) Implement Physical Model
- 3) Implement Auto-tuning
- 4) Implement extended PID loop
- 5) Implement particle base state estimation
- 6) Refine Physical model to include gaussian acceleration profiles
- 7) Refine Physical model to include angular momentum
- 8) Refine state estimation to include slip and non slip
- 9) Refine physical model to include slip and non-slip
- 10) Use maximum entropy state estimation and extended PID loops to do field control
- 11) Safety authorization mechanism

Appendix

XXXX

		<u>Wheel</u> <u>Size</u>	<u>Design</u> <u>Payload</u>	<u>Power</u>	<u>Resoluton</u>	<u>Notes</u>
Small		xx	10kg	60W	TBC	-xx
	xx	xx	100kg	180-250	100µm	-xx
	xx	xx	100kg	180-250	100µm	-xx
	xx	xx	100kg	180-250	100µm	-xx
Large		xx	500-1000kg	180-250	100µm	-xx

xxx
xxx

Our Current Flagship Magni 5th Generation

Fully certified robot in production - sold since Aug 2018

Payload: 100kg

Localization &
Navigation

Mobility

Power

AI: Object
awareness

Compute
infrastructure

